UNIVERSIDADE FEDERAL DE ITAJUBÁ

ECOX21 - MARATONA DE PROGRAMAÇÃO I

Profs. Edmilson Marmo e Luiz Olmes



Simulado Geral 2

14/10/2026

Regras

1. Há 12 problemas que devem ser resolvidos no tempo estipulado.
2. Os dados de entrada devem ser lidos a partir da entrada padrão.
3. Os dados de saída devem ser escritos na saída padrão.
4. Quando uma linha contém vários valores, eles estarão separados por um único espaço. Não há espaços extras na entrada/saída. Não há linhas em branco na entrada.
5. A entrada e a saída usam o alfabeto latino. Não haverá letras com til, acentos, tremas ou outros sinais diacríticos.
6. Todas as linhas da entrada e da saída, incluindo a última contêm o caractere de fim de linha.
7. O código fonte de cada solução deve ser enviado pelo Boca: `boca.unifei.edu.br`

Ambiente de Testes

A correção das soluções enviadas é realizada no sistema operacional Red Hat Enterprise Linux, versão 8.6 (Ootpa), usando os seguintes compiladores/interpretadores:

```
C: gcc versão 8.5.0 20210514 (Red Hat 8.5.0-10)
C++: g++ versão 8.5.0 20210514 (Red Hat 8.5.0-10)
Java: openjdk versão 1.8.0_342
Python: python3 versão 3.6.8
```

Limites

```
Memória (C, C++, Python): 1GB
Memória (Java): 1GB + 100MB stack
Tamanho máximo do código fonte: 100KB
Tamanho máximo do arquivo executável: 1MB
```

Códigos que extrapolem os limites permitidos receberão *Runtime Error* como resposta.

Comandos de Compilação

```
C: gcc -g -O2 -std=gnu11 -static -lm
C++: g++ -g -O2 -std=gnu++17 -static -lm
Java: javac
```

Códigos C/C++

- O programa deve retornar zero, executando, como último comando, `return 0` ou `exit(0)`.
- Para entradas grandes, objetos `iostream` podem ser lentos, devido a questões de sincronização de buffer com a biblioteca `stdio`. Recomenda-se desabilitar este mecanismo de sincronização em programas que empregam `std::cin` e `std::cout` através dos seguintes comandos:

  ```
  std::ios::sync_with_stdio(0);
  std::cin.tie(0);
  ```

  Note que, neste caso, deve-se evitar usar `printf` e `scanf` no mesmo programa, pois a separação dos buffers pode levar a resultados inesperados.

Códigos Java

- O programa não deve estar encapsulado em um `package`.
- Para cada problema, o arquivo `.java` e a `public class` devem ter o mesmo nome `basename` mostrado no Boca.
- Comando de execução: `java -Xms1024m -Xmx1024m -Xss100m`

Códigos Python

- Apenas Python 3 é suportado. Python 3 não é compatível com Python 2.
- **Atenção**: não é garantido que soluções escritas em Python executarão dentro do tempo limite especificado para cada problema.
- Comando de execução: `python3`

Problema $\mathcal{A}$

MAPEAMENTO DA ÁREA

Autoria: Edmilson Marmo
Timelimit: 1.0s

Um robô explorador foi programado para catalogar posições em uma região representada por um plano cartesiano. A região é dividida por uma fronteira retilínea, e o robô não pode utilizar posições localizadas exatamente sobre essa fronteira.

Para organizar as posições disponíveis, o robô utiliza um sistema de numeração. Inicialmente, são definidos os eixos x e y e uma equação linear que representa a fronteira. Em seguida, os pontos de coordenadas inteiras do plano são percorridos seguindo uma espiral no sentido horário.

O número 0 é atribuído ao ponto $(0, 0)$ e, a partir dele, os números seguintes são atribuídos aos demais pontos encontrados durante o percurso da espiral. Entretanto, sempre que um ponto estiver localizado exatamente sobre a fronteira, ele é ignorado e não recebe número.

Caso o ponto $(0, 0)$ esteja sobre a fronteira, o número 0 é atribuído ao primeiro ponto que não pertença à fronteira, respeitando a mesma ordem de percurso da espiral.

A Figura 1 apresenta a ordem de percurso dos pontos quando nenhum deles é descartado.

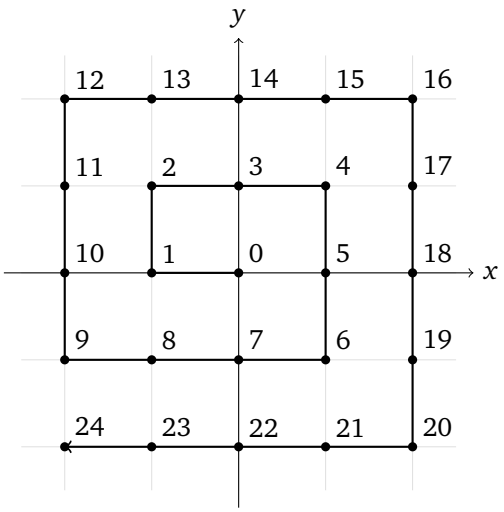


Figura 1: Ordem de percurso dos pontos de coordenadas inteiras.

Dessa forma, cada número utilizado pelo robô identifica uma única posição válida da região.

Durante uma missão, é necessário determinar se duas posições catalogadas estão localizadas do mesmo lado da fronteira ou em lados opostos da fronteira. Sua tarefa é responder a essas consultas.

Entrada

A primeira linha da entrada contém um inteiro T ($1 \leq T \leq 100$), representando a quantidade de casos de teste. A primeira linha de cada caso de teste contém dois inteiros a e b ($-5 \leq a \leq 5$ e $-10 \leq b \leq 10$), que definem a equação da fronteira:

$$y = ax + b.$$

A segunda linha de cada caso de teste contém um inteiro K ($1 \leq K \leq 1000$), indicando a quantidade de consultas. Cada uma das K linhas seguintes contém dois inteiros M e N ($0 \leq M, N \leq 65535$), representando duas posições segundo o sistema de numeração utilizado pelo robô.

Saída

Para cada caso de teste, seu programa deve produzir $K + 1$ linhas.

A primeira linha deve conter a identificação do caso de teste no formato:

`Caso X`

onde `X` deve ser substituído pelo número do caso de teste, iniciando em 1.

Para cada consulta, imprima

`MESMO LADO`

se as duas posições estiverem do mesmo lado da barreira, ou

`LADOS OPOSTOS`

caso estejam em lados opostos.

Exemplos

Entrada	*Saída*
2 1 2 10 26 25 25 11 24 9 23 28 25 9 25 1 25 0 9 1 23 12 26 17 1 2 12 0 1 1 2 2 3 3 4 4 5 5 6 6 7 7 8 8 9 9 10 10 11 11 12	Caso 1 MESMO LADO MESMO LADO MESMO LADO MESMO LADO MESMO LADO LADOS OPOSTOS LADOS OPOSTOS LADOS OPOSTOS LADOS OPOSTOS LADOS OPOSTOS Caso 2 MESMO LADO MESMO LADO MESMO LADO MESMO LADO MESMO LADO MESMO LADO MESMO LADO MESMO LADO LADOS OPOSTOS MESMO LADO MESMO LADO LADOS OPOSTOS

Problema $\mathcal{B}$

O PROBLEMA 3N+1

Timelimit: 1.0s

Durante um estudo experimental sobre sequências de números inteiros, uma pesquisadora utiliza um processo que, a partir de um inteiro positivo n, gera novos valores de acordo com as seguintes regras:

- se n for par, o próximo valor será $\frac{n}{2}$;
- se n for ímpar, o próximo valor será $3n + 1$.

O processo é repetido utilizando o novo valor obtido e termina quando o valor 1 é alcançado.

Por exemplo, iniciando com $n = 22$, é produzida a sequência

$$22,\ 11,\ 34,\ 17,\ 52,\ 26,\ 13,\ 40,\ 20,\ 10,\ 5,\ 16,\ 8,\ 4,\ 2,\ 1.$$

Define-se o **comprimento da sequência** de um número n como a quantidade de valores produzidos desde o valor inicial até o valor 1, incluindo ambos.

Assim, no exemplo anterior, o comprimento da sequência iniciada em 22 é igual a 16.

A pesquisadora deseja analisar todos os números pertencentes a determinados intervalos. Para cada intervalo, ela precisa determinar o maior comprimento de sequência obtido entre todos os números nele contidos. Sua tarefa é realizar essa análise.

Entrada

A primeira linha da entrada contém um inteiro T, representando a quantidade de casos de teste. Cada uma das T linhas seguintes contém dois inteiros positivos i e j ($0 < i, j < 1\,000\,000$), representando os extremos do intervalo que deve ser analisado. O primeiro valor informado não é necessariamente menor que o segundo.

Saída

Para cada caso de teste, imprima uma linha contendo três inteiros. Os dois primeiros devem ser i e j, exatamente na mesma ordem em que foram fornecidos na entrada. O terceiro deve ser o maior comprimento de sequência encontrado considerando todos os números entre i e j, inclusive. Os três valores devem ser separados por um único espaço.

Exemplos

Entrada	*Saída*
1 10 100 200 201 210 900 1000	1 10 20 100 200 125 201 210 89 900 1000 174

Problema C

ELEIÇÃO DO REPRESENTANTE

Autoria: Edmilson Marmo
Timelimit: 1.0s

Em uma eleição para escolher o representante dos estudantes, cada eleitor deve ordenar todos os candidatos de acordo com sua preferência. O primeiro candidato da lista representa sua primeira preferência, o segundo representa sua segunda preferência, e assim sucessivamente.

Em cada etapa da eleição, o voto de cada eleitor é atribuído ao candidato mais bem posicionado em sua lista de preferências que ainda não tenha sido eliminado.

Na primeira contagem, como nenhum candidato foi eliminado, cada voto é atribuído à primeira preferência do respectivo eleitor. Se algum candidato receber mais de 50% dos votos, ele será eleito.

Caso nenhum candidato obtenha mais de 50% dos votos, todos os candidatos empatados com a menor quantidade de votos são eliminados. Em seguida, uma nova contagem é realizada. Nessa nova contagem, cada voto é atribuído ao candidato ainda não eliminado mais bem posicionado na respectiva lista de preferências.

Esse processo de contagem e eliminação é repetido até que um candidato obtenha mais de 50% dos votos ou até que todos os candidatos que permanecem na eleição estejam empatados.

Entrada

A primeira linha da entrada contém um inteiro positivo T, indicando a quantidade de casos de teste.

Essa linha é seguida por uma linha em branco. Há também uma linha em branco entre dois casos de teste consecutivos.

A primeira linha de cada caso de teste contém um inteiro n ($1 \leq n \leq 20$), indicando a quantidade de candidatos.

As n linhas seguintes contêm os nomes dos candidatos, na ordem correspondente à sua numeração. O primeiro candidato recebe o número 1, o segundo recebe o número 2, e assim sucessivamente. Cada nome possui no máximo 80 caracteres e pode conter quaisquer caracteres imprimíveis.

Em seguida, são fornecidos até 1000 votos, um por linha.

Cada voto contém os números de 1 a n, sem repetição, indicando a ordem de preferência de um eleitor. O primeiro número representa sua primeira opção, o segundo número representa sua segunda opção e assim sucessivamente.

A lista de votos de um caso de teste termina quando uma linha em branco é encontrada ou quando a entrada termina.

Saída

Para cada caso de teste, imprima o nome do candidato vencedor.

Caso a eleição termine com todos os candidatos restantes empatados, imprima o nome de cada um desses candidatos, um por linha, na mesma ordem em que foram apresentados na entrada.

A saída de dois casos de teste consecutivos deve ser separada por uma linha em branco.

Exemplos

Entrada	*Saída*
1 3 Joao Dot Maria Silva Karina Cinq 1 2 3 2 1 3 2 3 1 1 2 3 3 1 2	Joao Dot

Problema $\mathcal{D}$

ENTREGA NO CAMPUS

Autoria: Edmilson Marmo
Timelimit: 1.0s

Durante a organização de um evento universitário, um estudante ficou responsável por entregar materiais em diversos prédios localizados ao longo de uma mesma avenida do campus.

Para simplificar o planejamento, cada prédio é representado por uma posição inteira ao longo da avenida. A distância entre dois prédios é dada pelo valor absoluto da diferença entre suas posições.

O estudante deve escolher um dos prédios como ponto de partida, passar por todos os prédios que precisam receber materiais e, ao final das entregas, retornar ao ponto de partida.

Como o percurso será realizado a pé, ele deseja caminhar o mínimo possível.

Sua tarefa é determinar a menor distância total que o estudante precisa percorrer para realizar todas as entregas e retornar ao ponto de partida.

Durante a organização de um evento universitário, um estudante ficou responsável por entregar materiais em diversos prédios localizados ao longo de uma mesma avenida do campus.

Para simplificar o planejamento, cada prédio é representado por uma posição inteira ao longo da avenida. A distância entre dois prédios é dada pelo valor absoluto da diferença entre suas posições.

O estudante deve escolher um dos prédios como ponto de partida, passar por todos os prédios que precisam receber materiais e, ao final das entregas, retornar ao ponto de partida.

Como o percurso será realizado a pé, ele deseja caminhar o mínimo possível.

Sua tarefa é determinar a menor distância total que o estudante precisa percorrer para realizar todas as entregas e retornar ao ponto de partida.

Entrada

A primeira linha da entrada contém um inteiro T, indicando a quantidade de casos de teste.

Cada caso de teste é composto por duas linhas. A primeira contém um inteiro n ($1 \leq n \leq 20$), indicando a quantidade de prédios que devem ser visitados.

A segunda linha contém n inteiros distintos $x_1, x_2, \ldots, x_n$ ($0 \leq x_i \leq 99$), indicando as posições dos prédios ao longo da avenida.

Saída

Para cada caso de teste, imprima uma linha contendo um único inteiro, correspondente à menor distância total que o estudante precisa percorrer para visitar todos os prédios e retornar ao ponto de partida.

Exemplos

Entrada	*Saída*
2 4 24 13 89 37 6 7 30 41 14 39 42	152 70

Problema $\mathcal{E}$

GUERRA EM MARTE

Autoria: Edmilson Marmo
Timelimit: 1.0s

A guerra em Marte é diferente da guerra na Terra. Como a população marciana é muito pequena, eles fazem a guerra utilizando um sistema computacional para determinar o vencedor, ou os vencedores, do conflito.

O sistema faz com que cada participante da guerra apresente duas palavras, cada uma com comprimento máximo de 100 caracteres, contendo somente letras minúsculas de `a` a `z`. Na verdade, o alfabeto marciano é bem diferente, mas estamos utilizando um processo de tradução para a Terra.

Para cada palavra, o sistema calcula a soma dos valores de seus caracteres, utilizando a seguinte tabela de conversão:

letra	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
valor	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5

A partir do total obtido para cada palavra, é calculado o resto da divisão por 4. Os dois valores resultantes são utilizados, respectivamente, como os parâmetros da função de Ackermann:

$$A(m,n) = \begin{cases} n+1, & \text{se } m = 0, \\ A(m-1,1), & \text{se } m > 0 \text{ e } n = 0, \\ A(m-1, A(m,n-1)), & \text{se } m > 0 \text{ e } n > 0. \end{cases}$$

O valor obtido pela função de Ackermann corresponde à pontuação do participante.

Como os marcianos não são bons programadores, eles solicitaram que você desenvolva um programa para determinar qual participante, ou quais participantes, obteve a maior pontuação.

Entrada

A entrada contém diversos casos de teste. A primeira linha de cada caso de teste contém um inteiro N ($1 \leq N \leq 100$), representando o número de participantes.

Para cada participante são fornecidas duas palavras, uma em cada linha, correspondentes aos dois parâmetros utilizados no cálculo da função de Ackermann. Cada palavra possui no máximo 100 caracteres, contém apenas letras minúsculas de `a` a `z` e não possui espaços em branco.

Os participantes são numerados de 1 a N, de acordo com a ordem em que suas palavras aparecem na entrada. A entrada termina quando $N = 0$. Esse valor não representa um caso de teste.

Saída

Para cada caso de teste, imprima uma única linha contendo

```
VENCEDOR: X
```

onde `X` representa o número do participante que obteve a maior pontuação.

Caso haja mais de um vencedor, seus números devem ser apresentados em ordem crescente, separados por - (um espaço, um traço e um espaço).

Exemplos

Entrada	*Saída*
2 aa aa an aa 5 aadfjdilkgrijgjdfjfdgif sdofgfrgofgpoub aaa asfgrgtr fdgrghth sdfrtgr edrertzz dgfrygr zdfvezz retrfyh 0	VENCEDOR: 2 VENCEDOR: 1 - 3

Problema $\mathcal{F}$

PAINEL

Autoria: Edmilson Marmo
Timelimit: 1.0s

No Trabalho Final de Graduação do curso de Engenharia de Computação, uma das etapas consiste no Workshop de Trabalhos em Andamento. Para essa atividade, cada equipe deve preparar um painel apresentando o estado atual de seu trabalho.

Neste ano, buscando incentivar a originalidade e a criatividade dos alunos, a coordenação estabeleceu que os painéis deverão conter apenas duas imagens. As imagens são retangulares e devem ser colocadas com seus lados paralelos aos lados do painel. Além disso, cada imagem deve ficar inteiramente contida no painel e as duas imagens não podem se sobrepor.

As imagens podem ser rotacionadas em 90 graus antes de serem colocadas no painel. Dependendo das dimensões do painel e das duas imagens, pode não ser possível posicioná-las respeitando todas essas condições.

Sua tarefa é determinar, dadas as dimensões do painel e das duas imagens, se é possível colocar ambas no painel.

Entrada

A entrada contém diversos casos de teste. A primeira linha de cada caso de teste contém dois inteiros X e Y ($0 \leq X, Y \leq 1000$), indicando, respectivamente, a largura e a altura do painel.

Cada uma das duas linhas seguintes contém dois inteiros L e H ($1 \leq L, H \leq 1000$), indicando, respectivamente, a largura e a altura de uma das imagens. A entrada termina quando $X = Y = 0$. Esse par de valores não representa um caso de teste.

Saída

Para cada caso de teste, imprima uma única linha contendo o caractere `S`, se for possível colocar as duas imagens no painel respeitando todas as condições descritas, ou o caractere `N`, caso contrário.

Exemplos

Entrada	*Saída*
7 5 3 4 3 4 10 10 6 6 6 6 0 0	S N

Problema $\mathcal{G}$

QUEBRA-CABEÇA DESLIZANTE

Timelimit: 1.0s

O quebra-cabeça de 15 peças é formado por um tabuleiro 4×4 contendo 15 peças numeradas de 1 a 15 e uma posição vazia. As peças podem deslizar pelo tabuleiro utilizando essa posição vazia.

O objetivo é, a partir de uma configuração inicial, organizar as peças na seguinte configuração final:

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	0

O valor 0 representa a posição vazia.

A única operação permitida consiste em trocar a posição vazia com uma peça localizada imediatamente acima, abaixo, à esquerda ou à direita dela.

Os movimentos são representados pelas letras:

- `D`: a posição vazia move-se uma posição para a direita;
- `E`: a posição vazia move-se uma posição para a esquerda;
- `C`: a posição vazia move-se uma posição para cima;
- `B`: a posição vazia move-se uma posição para baixo.

Por exemplo, se uma peça estiver imediatamente à direita da posição vazia, um movimento `D` troca essa peça com a posição vazia. Portanto, as letras indicam sempre o movimento realizado pela **posição vazia**.

Dada uma configuração inicial do quebra-cabeça, sua tarefa é determinar o número mínimo de movimentos necessários para atingir a configuração final ou identificar que isso não é possível.

Entrada

A primeira linha da entrada contém um inteiro T, indicando a quantidade de casos de teste. Cada caso de teste é composto por quatro linhas. Cada linha contém quatro inteiros, descrevendo uma linha do tabuleiro.

Os números de 1 a 15 representam as peças e o valor 0 representa a posição vazia.

Cada número de 0 a 15 aparece exatamente uma vez em cada caso de teste.

Saída

Para cada caso de teste, imprima uma única linha contendo o número mínimo de movimentos necessários para transformar a configuração inicial na configuração final.

Se essa transformação não for possível, imprima

```
IMPOSSIVEL
```

Caso o tabuleiro fornecido na entrada já esteja na configuração final, a resposta deverá ser 0.

Exemplos

Entrada	*Saída*
2 2 3 4 0 1 5 7 8 9 6 10 12 13 14 11 15 13 1 2 4 5 0 3 7 9 6 10 12 15 8 11 14	9 IMPOSSIVEL

Problema $\mathcal{H}$

TRANSPORTES

Timelimit: 1.0s

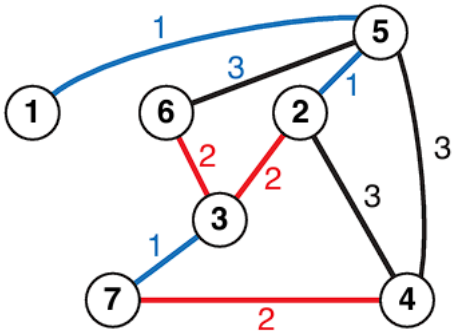
Ênia, a atual rainha da Nlogônia, é fascinada por meios de transporte de todos os tipos. Como consequência, a Nlogônia possui a melhor rede de transporte público do mundo inteiro.

Existem N estações espalhadas por todo o reino, K sistemas de transporte (metrô, ônibus, avião, barco, etc) e M ligações diretas entre pares de estações. Cada ligação pertence a um único sistema de transporte entre os K existentes, e pode ser percorrida nas duas direções. Para cada par de estações, existe no máximo uma ligação no total (ou seja, entre todos os sistemas).

Para usar os sistemas de transporte, o usuário precisa comprar passagens. Ênia estabeleceu as seguintes regras para estimular o transporte público:

- Cada um dos K sistemas de transporte tem um preço fixo de passagem. Assim, o preço de uma passagem do sistema i é P_i, independente da estação.
- A passagem só deve ser paga quando o usuário entrar no sistema de transporte.
- O usuário só pode entrar em um sistema por vez (por exemplo, para entrar no sistema de avião ele precisa sair do sistema de metrô).
- Após pagar o preço P_i para entrar no sistema i, o usuário pode se locomover o quanto desejar usando ligações do sistema i, contanto que ele não saia desse sistema. Caso o usuário troque de sistema, ele deverá pagar por uma nova passagem de preço P_i para retornar ao sistema i.

Por exemplo, considere a seguinte rede de transportes:



Note que há 7 estações e 9 ligações, das quais 3 são do sistema 1, 3 são do sistema 2 e 3 são do sistema 3. Suponha que os preços das passagens sejam $P_1 = 5$, $P_2 = 1$ e $P_3 = 7$ e que o usuário deseja fazer uma viagem da estação 1 para a estação 7, o menor custo total entre todas as possíveis viagens é 11:

- $1 \to 5$: usa o sistema 1, então pagamos $P_1 = 5$.
- $5 \to 2$: também usa o sistema 1, então não pagamos nada.
- $2 \to 3$: usa o sistema 2, então pagamos $P_2 = 1$ para trocar de sistema.
- $3 \to 7$: usa o sistema 1, então pagamos $P_1 = 5$ para voltar ao sistema 1.
- No total, essa viagem custará: 5 + 1 + 5 = 11.

Ênia deseja saber o menor custo total para ir da estação A para a estação B, e para isso te contratou. Responda corretamente, ou ela te expulsará da Nlogônia!

Entrada

A primeira linha da entrada contém três inteiros N, M e K ($2 \leq N \leq 10^5; 1 \leq M, K \leq 10^5$), representando o número de estações, o número de ligações entre as estações, e a quantidade de sistemas de transporte, respectivamente.

A segunda linha contém K números inteiros. O i-ésimo desses números é P_i ($1 \leq P_i \leq 1000; 1 \leq i \leq K$), representando o preço da passagem do sistema de transporte i. As próximas M linhas contém a descrição das ligações. A j-ésima dessas linhas tem três números V_j, U_j e T_j ($1 \leq V_j, U_j \leq N; 1 \leq j \leq M; V_j \neq U_j; 1 \leq T_j \leq K$), indicando que há uma ligação entre as estações V_j e U_j usando o sistema T_j.

A última linha da entrada contém dois números inteiros, A e B ($1 \leq A, B \leq N; A \neq B$), representando a estação inicial e a estação final da viagem requisitada pela rainha.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o menor custo total de uma viagem que começa na estação A e termina na estação B. Se não for possível fazer uma viagem de A a B usando a rede de transportes, imprima `-1`.

Exemplos

Entrada	*Saída*
6 4 1 50 1 4 1 6 1 1 3 2 1 4 6 1 3 5	-1

Entrada	*Saída*
5 5 2 1 1 1 2 1 5 2 1 1 3 2 4 3 1 4 5 2 2 3	2

Problema *I*

ELEMENTO X

Timelimit: 1.0s

Açúcar, tempero, e tudo que há de bom. Esses foram os ingredientes escolhidos para criar a sobremesa perfeita. Mas o professor Utônio, acidentalmente, acrescentou um ingrediente extra na mistura: o Elemento X!



Diferentemente de um famoso desenho animado, a adição do Elemento X não deu certo, e a sobremesa ficou completamente sem gosto. Como bons viajantes do tempo que somos, nossa tarefa é impedir essa catástrofe culinária.

A organização do laboratório do professor é incrivelmente eficiente e simples: o recipiente de cada um dos quatro ingredientes (colocados em ordem: r_1, r_2, r_3, r_4, com distância igual a 1 entre eles) é rotulado com a distância entre ele e o recipiente do Elemento X. Ao chegarmos do futuro, porém, descobrimos o motivo do acidente: o pote do Elemento X está com erro no rótulo, e mostra um valor maior que zero, o que não faz o menor sentido! Nossa tarefa é identificar qual é o recipiente com rótulo danificado e informar o professor antes que um desastre culinário ocorra!

Entrada

A entrada é composta por diversos casos de teste. Cada caso de teste é composto por uma única linha contendo quatro inteiros r_1, r_2, r_3, r_4, cada um representando a distância do respectivo pote para o recipiente do Elemento X. A entrada termina com fim de arquivo (`EOF`).

Saída

Para cada caso de teste, a saída deve conter um único inteiro $1 \leq i \leq 4$, que representa a posição do pote que contém o Elemento X, contada da esquerda para a direita.

Exemplos

Entrada	*Saída*
1 1 2 3 2 1 2 1	1 3

Problema $\mathcal{J}$

DESVIO DE RUA

Timelimit: 1.0s

A prefeitura de uma grande cidade da Nlogônia iniciou um programa de recuperação do asfalto de suas ruas. Na Nlogônia, cada rua liga diretamente dois cruzamentos, e pode ter mão única ou mão dupla. Por determinação de um antigo decreto real, sempre existe ao menos um caminho entre dois pontos quaisquer da cidade.

No programa de recuperação, uma única rua será recuperada por vez, e para isso a rua será fechada para o tráfego. Esse fechamento pode causar caos no trânsito local ao violar o decreto real, impedindo vários cidadãos de voltarem para casa dos seus trabalhos e vice-versa. A prefeitura pode converter algumas das ruas de mão única em mão dupla, mas prefere evitá-lo pois ruas de mão dupla tendem a causar acidentes mais graves; a prefeitura prefere criar desvios apenas invertendo as mãos das ruas de mão única já existentes. O Rei da Nlogônia solicitou seus préstimos para escrever um programa que, dada a descrição das ruas de uma cidade, determine se, quando uma dada rua é interditada para recuperação, continua existindo um caminho entre entre quaisquer dois pontos da cidade, mesmo que seja necessário alterar as mãos de direção de outras ruas.

Entrada

A entrada é composta por diversos casos de teste. A primeira linha de um caso de teste contém dois inteiros N e M ($1 \leq N \leq 10^3$; $1 \leq M \leq 10^5$), representando respectivamente o número de cruzamentos e o número de ruas da cidade. Os cruzamentos são identificados por inteiros de 1 a N e as ruas são identificadas por números inteiros de 1 a M. Cada uma das M linhas seguintes descreve uma rua e contém três inteiros A, B e T ($1 \leq A, B \leq N$), onde A e B são os cruzamentos que a rua liga diretamente, e T indica a mão de direção da rua: se $T = 1$ a rua tem mão única na direção de A para B; se $T = 2$, a rua tem mão dupla. A primeira rua descrita será interditada para recuperação.

Saída

Para cada caso de teste seu programa deve imprimir uma linha contendo um caractere que descreve o que a prefeitura deve fazer para respeitar o decreto real após o fechamento da rua para reformas:

- '–': não é necessário qualquer tipo de alteração nas outras ruas.
- '*': é impossível respeitar o decreto real, independente de quaisquer mudanças nas outras ruas.
- '1': é possível cumprir o decreto real apenas invertendo as mãos de algumas das ruas de mão única.
- '2': é possível cumprir o decreto real, mas é necessário converter algumas ruas de mão única para mão dupla.

Exemplos

Entrada	*Saída*
3 3 1 2 2 2 3 2 3 1 2 3 3 1 2 1 2 3 1 3 1 1 2 1 1 2 2 5 6 3 4 1 1 3 2 2 4 2 3 5 2 2 1 1 4 1 1	- 2 * 1

Problema $\mathcal{K}$

FIGURINHAS

Timelimit: 1.0s

Beto e Zeca colecionam figurinhas. Cada figurinha é identificada por um número inteiro, porém, os garotos podem ter figurinhas repetidas em suas coleções. Os meninos querem saber se é possível escolher N figurinhas de Beto e M figurinhas de Zeca, tal que o número de qualquer uma das figurinhas de Beto seja estritamente menor que o número de qualquer uma das figurinhas de Zeca.

Entrada

A entrada contém vários casos de teste. A primeira linha de um caso de teste contém dois inteiros B e Z, indicando quantas figurinhas Beto e Zeca possuem, respectivamente ($1 \leq B, Z \leq 10^5$). A segunda linha de um caso de teste contém os inteiros N e M, como descritos ($1 \leq N \leq B; 1 \leq M \leq Z$). A terceira linha contém B inteiros b_i, que são as figurinhas de Beto ($-10^9 \leq b_i \leq 10^9$). A quarta linha contém Z inteiros z_i, que são as figurinhas de Zeca ($-10^9 \leq z_i \leq 10^9$). Valores $B = Z = 0$ indicam o fim da entrada.

Saída

Para cada caso de teste, imprima apenas as palavras `YES` ou `NO`, caso seja possível, ou não, satisfazer as condições do problema.

Exemplos

Entrada	*Saída*
3 3 2 1 1 2 3 3 4 5 3 3 3 3 1 2 3 3 4 5 5 2 3 1 1 1 1 1 1 2 2 0 0	YES NO YES

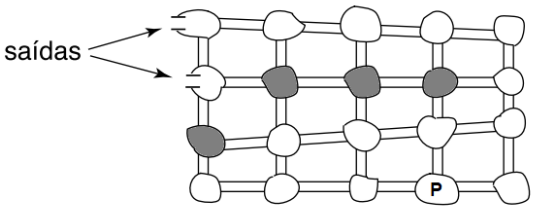
Problema $\mathcal{L}$

FUGA DA PRINCESA

Timelimit: 1.0s

Em um passado recente, os irmãos Mário e Luigi derrotaram o vilão Bowser Koopa e salvaram a princesa Toadstool, devolvendo a paz para a terra dos dinossauros. Entretanto, o vilão Bowser apenas fingiu sua derrota e ficou à espreita, só esperando os irmãos saírem de férias para retornar à ativa. Desta vez, com a ajuda de seus filhos, Bowser capturou a princesa e a prendeu em uma caverna. A caverna é formada por várias câmaras interligadas entre si, como uma grade retangular de M linhas e N colunas. Existem guardas em alguns salões da caverna, porém, a princesa percebeu que várias câmaras não são vigiadas, pois a maioria dos seguidores do Bowser foi derrotada por Mário e Luigi e ele ainda não conseguiu criar um novo exército. Assim, na calada da noite, a princesa planeja uma fuga secreta, porém, ela não pode correr o risco de entrar em uma câmara vigiada, pois senão será presa pelos guardas.

A figura abaixo mostra uma caverna com 4 linhas e 5 colunas de câmaras, onde as câmaras em cinza são aquelas que possuem guardas; e a câmara com a letra P mostra onde está a princesa. A caverna é representada por uma matriz, cujos elementos descrevem as câmaras. Uma câmara que não contém guardas e possui uma saída para o exterior é representada pela letra S. Uma câmara vazia é representada pela letra E. Uma câmara que contém guardas é representada pela letra G. A câmara em que a princesa se encontra possui a letra P. A matriz do exemplo abaixo traz a representação da caverna mostrada na figura.



S	E	E	E	E
S	G	G	G	E
G	E	E	E	E
E	E	E	P	E

Entrada

A entrada contém vários casos de teste. A primeira linha de um caso de teste possui dois inteiros M ($1 \leq M \leq 10$) e N ($1 \leq N \leq 10$), que indicam a quantidade de linhas e colunas da caverna. As M linhas seguintes descrevem as câmaras, onde cada linha contém N caracteres. Valores $M = N = 0$ indicam o fim da entrada.

Saída

Para cada caso de teste, imprima um único número inteiro, indicando a quantidade mínima de câmaras pelas quais a princesa deve passar antes de conseguir sair da caverna (não contando a câmara em que ela está inicialmente, mas contando a câmara que tem saída para o exterior).

Exemplos

Entrada	*Saída*
4 5 SEEEE SGGGE GEEEE EEEPE 1 10 GSEEPEEESE 0 0	8 3